



moortgat.1 docs: add REPRODUCING_FIGURES.md mapping each paper figure to its script

1804387 · 1 hour ago

🕒 History

Preview

Code

Blame

277 lines (190 loc) · 7.59 KB

Raw



Reproducing the Figures

All commands are run from the `code/` subdirectory unless noted otherwise.

```
cd code
```



Overview

Figure generation is a two-step process:

1. **Data generation** — run validation, benchmark, or scan scripts that compute flash results and save them to `results/` as parquet or npz files.
2. **Plotting** — run lightweight plot scripts that load those saved results and write figures to `figures/`.

The sections below are organised by figure number and list both steps. The precomputed solution table (`results/CPA_ELV_all.parquet` , included in the repository) is required by most scripts — no additional setup is needed for it.

Main paper figures

Figure 1 — CO₂ + H₂O phase equilibria at 25, 150, and 250 °C

Data generation (~5 min):

```
python scripts/validate_co2h2o.py
```



Saves `results/validation_co2h2o.parquet` and `results/smooth_co2h2o.parquet`.

Plotting:

```
python scripts/plot_co2h2o_figures.py
```



Outputs per-temperature panels to `figures/co2h2o/T*.png`. Panels used in Fig. 1: `T298K.png` (25 °C), `T423K.png` (150 °C), `T523K.png` (250 °C).

Figure 2 — CO₂ solubility in NaCl brine at 40, 80, 120, and 250 °C

Data generation (~30 min):

```
python scripts/validate_co2nacl_full.py
```



Saves `results/ws_validation_co2nacl.parquet` and warm-start smooth-curve parquets (`results/ws2_smooth_co2h2o_ms*.parquet`).

Plotting:

```
python scripts/plot_co2nacl_figures.py
```



Outputs per-temperature panels to `figures/co2nacl/T*.png`. Panels used in Fig. 2: `T313K.png` (40 °C), `T353K.png` (80 °C), `T393K.png` (120 °C), `T523K.png` (250 °C).

Figure 3 — CO₂ + NaCl two-panel at 100 °C (x-CO₂ and y-CO₂)

Same data and script as Fig. 2. Panels used:

- `figures/co2nacl/T373K.png` (100 °C aqueous panel)
- `figures/co2nacl_ws/T373K.png` (100 °C CO₂-rich panel, from the same script run)

Figure 4 — eCPA flash results vs. CO₂ feed mole fraction z

Data generation: requires `results/scan_v4_table.npz` (see Figs. 7–9 below).

Plotting:

```
python scripts/plot_flash_vs_z.py
```



Output: `figures/flash_vs_z.png` .

Figure 5 — Mean SSI iteration count (CPA binary parameter-space scan)

Data generation — CPA parameter-space scan (~20 min):

```
python scripts/run_parameter_scan.py
```



Saves `results/scan_results_extended.npz` (86T × 18P × 19z grid).

Data generation — Newton polish scan (~10 min):

```
python scripts/run_newton_scan.py
```



Saves `results/scan_newton_results.npz` .

Plotting:

```
python scripts/plot_newton_figures.py
```



Output: `figures/scan/newton_heatmap.pdf` and `figures/scan/newton_heatmap.png` .

Figure 6 — SSI speedup ratio and best stability initial guess

Same prerequisites and script as Fig. 5.

Output: `figures/scan/strategy_comparison_bar.pdf` and `.png`.

Figures 7, 8, 9 — eCPA ternary composition and timing grids

Data generation — eCPA warm-start scan (~27 min, uses all CPU cores):

```
python scripts/run_warmstart_scan.py
```



Saves `results/scan_v4_table.npz` and `results/scan_v4_metrics.parquet`.

Plotting:

```
python scripts/plot_scan_figures.py
```



Outputs (in `figures/scan_v4/`):

- `ecpa_composition_aq_grid.pdf` — Fig. 7 (CO₂ mol% in aqueous phase)
- `ecpa_composition_c_grid.pdf` — Fig. 8 (H₂O mol% in CO₂-rich phase)
- `ecpa_timing_heatmap.pdf` — Fig. 9 (mean wall time per flash call)

Figures 10, 11, 12 — Reservoir simulator results

Run the simulator (~10 min on a laptop, 50×50 grid, 300 time steps):

```
python scripts/run_simulator_paper.py
```



Outputs (in `figures/simulator/`):

- `fig_compositions.png` — Fig. 10 (final spatial fields, CPA vs. eCPA)
- `fig_simulator.png` — Fig. 11 (spatial maps at multiple time steps)
- `fig_perf.png` — Fig. 12 (CO₂ trapping fractions over time)

Supplemental Information figures

Figure S1 — CO₂ + H₂O full temperature panel (35 isotherms, 0–350 °C)

Same data and script as Fig. 1. All panels in `figures/co2h2o/T*.png` are used.

Figure S2 — CO₂ + NaCl brine full temperature panel (moderate T)

Same data and script as Fig. 2. Panels from `figures/co2nacl/T*.png` for $T \leq 523$ K.

Figure S3 — CO₂ + NaCl brine at high temperatures (300 and 350 °C)

Same data as Fig. 2 (requires `validate_co2nacl_full.py`).

Plotting:

```
python scripts/plot_co2nacl_highT_figures.py
```



Outputs: `figures/co2nacl/T573K.png` and `figures/co2nacl/T623K.png`.

Figure S4 — AARE heatmap for CO₂ + NaCl (all T, all m_s)

Same data and script as Fig. 2. Output: `figures/validation_heatmap_extended.png`.

Figures S5 and S5b — Aqueous density parity plot and error surface

Data generation (~1 min):

```
python scripts/validate_density.py
```



Saves `results/pure_water_density_iapws.parquet`.

Plotting (integrated into the same script): Output: `figures/density/parity_ecpa.png` .

Figure S6 — ARE scatter plot and error heatmap (CO₂ + H₂O)

Same data and script as Fig. 1. Output: `figures/co2h2o/error_heatmap.png` .

Figure S7 — SSI + Newton polish iteration breakdown

Same prerequisites as Figs. 5–6 (requires `run_warmstart_scan.py`).

Plotting:

```
python scripts/plot_newton_stats.py
```



Output: `figures/scan_v4/ecpa_newton_stats.pdf` .

Figure S8 — CPA flash strategy comparison (all 9,825 two-phase conditions)

Same prerequisites and script as Figs. 5–6. The strategy comparison bar chart is generated alongside the Fig. 5 heatmap:

```
python scripts/plot_newton_figures.py
```



Output: `figures/scan/strategy_comparison_bar.pdf` .

Figure S9 — CO₂ trapping fractions over time

Same script as Figs. 10–12:

```
python scripts/run_simulator_paper.py
```



Output: `figures/simulator/fig_perf.png` .

Complete reproduction sequence

To regenerate every figure from scratch, run the following in order. Steps marked (*slow*) take more than a few minutes.

`cd code`



```
# 1. Build solution table (if not using the pre-built CPA_ELV_all.parqu
# python scripts/build_solution_table.py
```

```
# 2. CO2 + H2O binary validation
python scripts/validate_co2h2o.py
```

```
# 3. CO2 + NaCl ternary validation
python scripts/validate_co2nacl_full.py
```

```
# 4. Aqueous density validation
python scripts/validate_density.py
```

```
# 5. CPA binary parameter-space scan
python scripts/run_parameter_scan.py
```

```
# 6. CPA Newton polish scan
python scripts/run_newton_scan.py
```

```
# 7. eCPA warm-start scan (generates scan_v4_table.npz)
python scripts/run_warmstart_scan.py
```

```
# 8. Reservoir simulator
python scripts/run_simulator_paper.py
```

```
# — Plotting —
```

<code>python scripts/plot_co2h2o_figures.py</code>	<code># Figs. 1, S1, S6</code>
<code>python scripts/plot_co2nacl_figures.py</code>	<code># Figs. 2, 3, S2, S4</code>
<code>python scripts/plot_co2nacl_highT_figures.py</code>	<code># Fig. S3</code>
<code>python scripts/plot_flash_vs_z.py</code>	<code># Fig. 4</code>
<code>python scripts/plot_newton_figures.py</code>	<code># Figs. 5, 6, S8</code>
<code>python scripts/plot_scan_figures.py</code>	<code># Figs. 7, 8, 9</code>
<code>python scripts/plot_newton_stats.py</code>	<code># Fig. S7</code>
<code>python scripts/benchmark_simplified_flash.py</code>	<code># Figs. 11, 12</code>

